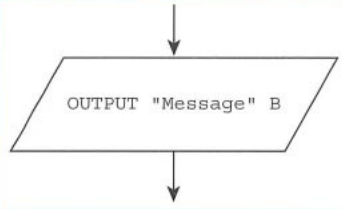
	Structured English	Pseudocode	Flowchart
Output	`OUTPUT "Message"` `OUTPUT B`	`OUTPUT "Message", B`	

Table 12.01 Constructs for computing algorithms

12.04 Variables

When we input data for a process, individual values need to be stored in memory. We need to be able to refer to a specific memory location so that we can write statements of what to do with the value stored there. We refer to these named memory locations as **variables**. You can imagine these variables like boxes with name labels on them. When a value is input, it is stored in the box with the specified name (identifier) on it.

KEY TERM

Variable: a storage location for a data value that has an identifier

For example, the variable used to store a count of how many guesses have been made in a number guessing game might be given the identifier `NumberOfGuesses` and the player's name might be stored in a variable called `ThisPlayer`, as shown in Figure 12.04.

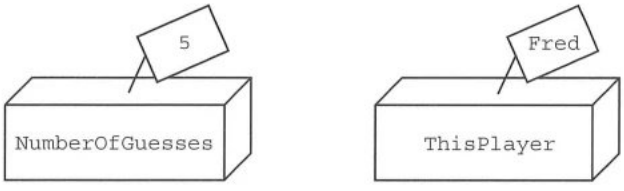


Figure 12.04 Variables

Variable identifiers should not contain spaces, only letters, digits and _ (the underscore symbol). To make algorithms easier to understand, the naming of a variable should reflect the variable's use. This means often that more than one word is used as an identifier. The formatting convention used here is known as CamelCaps. It makes an identifier easier to read.

12.05 Assignments

Assigning a value

The following pseudocode stores the value that is input (for example 15) in a variable with the identifier `Number` (see Figure 12.05(a)).

```
INPUT Number
```

The following pseudocode stores the value 1 in the variable with the identifier `NumberOfGuesses` (see Figure 12.05(b)).

```
NumberOfGuesses ← 1
```



Figure 12.05 Variables being assigned a value

Updating a value

The following pseudocode takes the value stored in `NumberOfGuesses` (see Figure 12.06 (a)), adds 1 to that value and then stores the new value back into the variable `NumberOfGuesses` (see Figure 12.06 (b)).

```
NumberOfGuesses ← NumberOfGuesses + 1
```

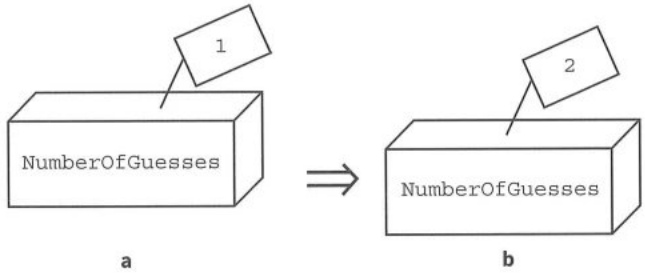


Figure 12.06 Updating the value of a variable

Copying a value

Values can be copied from one variable to another.

The following pseudocode takes the value stored in `Value1` and copies it to `Value2` (see Figure 12.07).

```
Value2 ← Value1
```

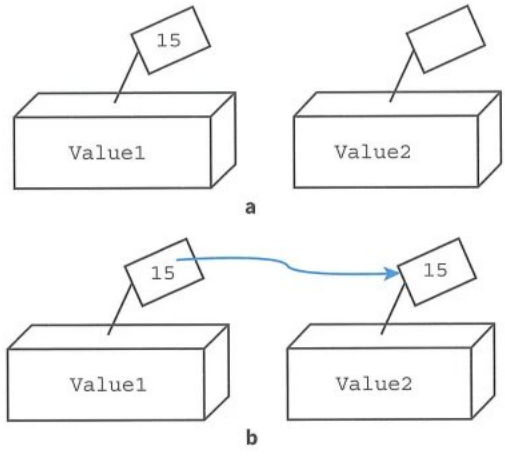


Figure 12.07 Copying the value of a variable

The value in `Value1` remains the same until it is assigned a different value.

Swapping two values

If we want to swap the contents of two variables, we need to store one of the values in another variable temporarily. Otherwise the second value to be moved will be overwritten by the first value to be moved.

In Figure 12.08(a), we copy the content from `Value1` into a temporary variable called `Temp`. Then we copy the content from `Value2` into `Value1` Figure 12.08(b)). Finally, we can copy the value from `Temp` into `Value2` (Figure 12.08(c)).

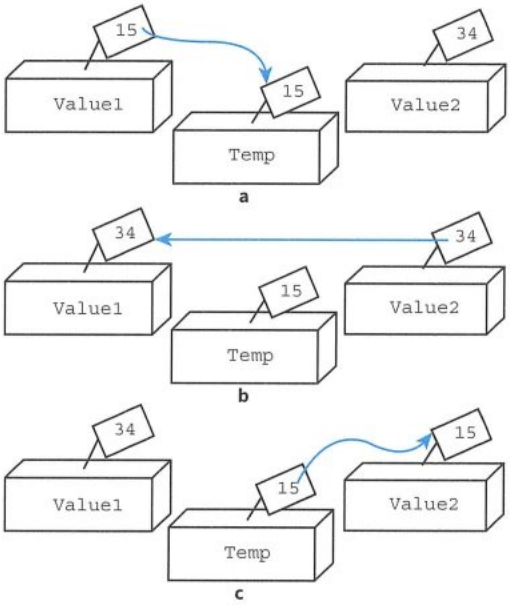


Figure 12.08 Swapping the values of two variables

Using pseudocode we write:

```
Temp ← Value1
Value1 ← Value2
Value2 ← Temp
```

WORKED EXAMPLE 12.01

Using input, output, assignment and sequence constructs

The problem to be solved: Convert a distance in miles and output the equivalent distance in km.

Step 1: Write the problem as a series of structured English statements:

```
INPUT number of miles
Calculate number of km
OUTPUT calculated result as km
```

(Continued)

Step 2: Analyse the data values that are needed.

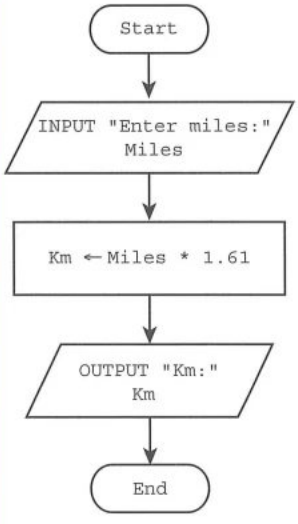
We need a variable to store the original distance in miles and a variable to store the result of multiplying the number of miles by 1.61. It is helpful to construct an **identifier table** to list the variables.

Identifier	Explanation
`Miles`	Distance as a whole number of miles
`Km`	The result from using the given formula: Km = Miles * 1.61

Table 12.02 Identifier table for miles to km conversion

Step 3: Provide more detail by drawing a flowchart or writing pseudocode.

The detail given in a flowchart should be the same as the detail given in pseudocode. It should use the basic constructs listed in Table 12.01.
Figure 12.09 represents our algorithm using a flowchart and the equivalent pseudocode.




```
INPUT "Enter miles:" Miles
Km ← Miles * 1.61
OUTPUT "km:", Km
```

Figure 12.09 Flowchart and pseudocode for miles to km conversion

KEY TERM

Identifier table: a table listing the variable identifiers required for the solution, with explanations and data types (see Chapter 13)

TASK 12.03

Consider the following algorithm steps.

1 Input a length in inches.
2 Calculate the equivalent in centimetres.
3 Output the result.

List the variables required in an identifier table.

Write pseudocode for the algorithm.

12.06 Logic statements

In Section 12.02, we looked at an algorithm with different steps depending on some other condition:

IF there are engineering works on the Victoria Line

THEN

Take the Piccadilly Line to Green Park (6 stations)

Take the Jubilee Line to Westminster (1 station)

ELSE

Take the Victoria Line to Green Park (4 stations)

Take the Jubilee Line to Westminster (1 station)

The selection construct in Table 12.01 uses a condition to follow either the first group of steps or the second group of steps (see Figure 12.10).




```
IF A < B
  THEN
    <statement(s)>
  ELSE
    <statement(s)>
ENDIF
```


Figure 12.10 Pseudocode for the selection construct

A condition consists of at least one logic proposition (see Chapter 4, Section 4.01). Logic propositions use the relational (comparison) operators shown in Table 12.03.

Operator	Comparison
=	Is equal to
<	Is less than
>	Is greater than
<=	Is less than or equal to
>=	Is greater than or equal to
<>	Is not equal to

Table 12.03 Relational operators

Conditions are either TRUE or FALSE. In pseudocode, we distinguish between the relational operator = (which tests for equality) and the assignment symbol ←.

A person is classed as a child if they are under 13 and as an adult if they are over 19. If they are between 13 and 19 inclusive they are classed as teenagers. We can write these statements as logic statements.

- If Age < 13 then person is a child.
- If Age > 19 then person is an adult.
- If Age >= 13 AND Age <= 19 then person is a teenager.

TASK 12.04

A town has a bus service where passengers under the age of 12 and over the age of 60 do not need to pay a fare. Write the logic statements for free fares.

A number-guessing game follows different steps depending on certain conditions. Here is a description of the algorithm.

- The player inputs a number to guess the secret number stored.
- If the guess was correct, output a congratulations message.
- If the number input was larger than the secret number, output message "secret number is smaller".
- If the number input was smaller than the secret number, output message "secret number is greater".

We can re-write the number-guessing game steps as an algorithm in pseudocode:

```
SET value for secret number
INPUT Guess
IF Guess = SecretNumber
  THEN
    OUTPUT "Well done. You have guessed the secret number"
  ELSE
    IF Guess > SecretNumber
      THEN
        OUTPUT "secret number is smaller"
      ELSE
        OUTPUT "secret number is greater"
    ENDIF
ENDIF
```


More complex conditions can be formed by using the logical operators AND, OR and NOT. For example, the number-guessing game might allow the player multiple guesses; if the player has not guessed the secret number after 10 guesses, a different message is output.

```
IF Guess = SecretNumber
  THEN
    OUTPUT "Well done. You have guessed the secret number"
  ELSE
    IF Guess > SecretNumber AND NumberofGuesses = 10
      THEN
        OUTPUT "You still have not guessed the secret number"
      ELSE
        IF Guess > SecretNumber
          THEN
            OUTPUT "secret number is smaller"
          ELSE
            OUTPUT "secret number is greater"
        ENDIF
    ENDIF
ENDIF
```

complex condition

WORKED EXAMPLE 12.02

Using selection constructs

The problem to be solved: Take three numbers as input and output the largest number.

There are several different methods (algorithms) to solve this problem. Here is one method.

1. Input all three numbers at the beginning.
2. Store each of the input values in a separate variable (the identifiers are shown in Table 12.04).
3. Compare the first number with the second number and then compare the bigger one of these with the third number.
4. The bigger number of this second comparison is output.

See Worked Example 12.03 for another solution.

Identifier	Explanation
`Number1`	The first number to be input
`Number2`	The second number to be input
`Number3`	The third number to be input

Table 12.04 Identifier table for biggest number problem

The algorithm can be expressed in the following pseudocode:

```
INPUT Number1
INPUT Number2
INPUT Number3
IF Number1 > Number2
  THEN
  // Number1 is bigger
    IF Number1 > Number3
      THEN
        OUTPUT Number1
      ELSE
        OUTPUT Number3
    ENDIF
  ELSE
  // Number2 is bigger
    IF Number2 > Number3
      THEN
        OUTPUT Number2
      ELSE
        OUTPUT Number3
    ENDIF
ENDIF
```


When an `IF` statement contains another `IF` statement, we refer to these as **nested `IF` statements**.

KEY TERM

Nested IF statements: conditional statements within conditional statements

Question 12.01

What changes do you need to make to output the smallest number?

WORKED EXAMPLE 12.03

Using selection constructs (alternative method)

The problem to be solved: Take three numbers as input and output the largest number.

This is an alternative method to Worked Example 12.02.

1. Input the first number and store it in `BiggestSoFar`
2. Input the second number and compare it with the value in `BiggestSoFar`.
3. If the second number is bigger, assign its value to `BiggestSoFar`
4. Input the third number and compare it with the value in `BiggestSoFar`
5. If the third number is bigger, assign its value to `BiggestSoFar`
6. The value stored in `BiggestSoFar` is output.

The identifiers required for this solution are shown in Table 12.05.

Identifier	Explanation
`BiggestSoFar`	Stores the biggest number input so far
`NextNumber`	The next number to be input

Table 12.05 Identifier table for the alternative solution to the biggest number problem

The algorithm can be expressed in the following pseudocode:

```
INPUT BiggestSoFar
INPUT NextNumber
IF NextNumber > BiggestSoFar
  THEN
    BiggestSoFar ← NextNumber
ENDIF
INPUT NextNumber
IF NextNumber > BiggestSoFar
  THEN
    BiggestSoFar ← NextNumber
ENDIF
OUTPUT BiggestSoFar
```

Note that when we input the third number in this method the second number gets overwritten as it is no longer needed.

There are several advantages of using the method in Worked Example 12.03 compared to the method in Worked Example 12.02.

- Only two variables are used.
- The conditional statements are not nested and do not have an ELSE part. This makes them easier to understand.
- This algorithm can be adapted more easily if further numbers are to be compared (see Worked Example 12.04).

The disadvantage of the method in Worked Example 12.03 compared to the method in Worked Example 12.02 is that there is more work involved with this algorithm. If the second

number is bigger than the first number, the value of `BiggestSoFar` has to be changed. If the third number is bigger than the value in `BiggestSoFar` then the value of `BiggestSoFar` has to be changed again. Depending on the input values, this could result in two extra assignment instructions being carried out.

12.07 Loops

Look at the pseudocode algorithm in Worked Example 12.03. The two `IF` statements are identical. To compare 10 numbers, we would need to write this statement nine times. Moreover, if the problem changed to having to compare, for example, 100 numbers, our algorithm would become very tedious. If we use a repetition construct (a loop) we can avoid writing the same lines of pseudocode over and over again.

WORKED EXAMPLE 12.04

Repetition using REPEAT...UNTIL

The problem to be solved: Take 10 numbers as input and output the largest number.

We need one further variable to store a counter, so that we know when we have compared 10 numbers.

Identifier	Explanation
`BiggestSoFar`	Stores the biggest number input so far
`NextNumber`	The next number to be input
`Counter`	Stores how many numbers have been input so far

Table 12.06 Identifier table for the biggest number problem using `REPEAT...UNTIL`

The algorithm can be expressed in the following pseudocode:

```
INPUT BiggestSoFar
Counter ← 1
REPEAT
    INPUT NextNumber
    Counter ← Counter + 1
    IF NextNumber > BiggestSoFar
      THEN
        BiggestSoFar ← NextNumber
    ENDIF
UNTIL Counter = 10
OUTPUT BiggestSoFar
```

Note that when we input the next number in this method the previous number gets overwritten as it is no longer needed.

Question 12.02

What changes do you need to make to the algorithm in Worked Example 12.04:

a to compare 100 numbers?

b to take as a first input the number of numbers to be compared?

There is another loop construct that does the counting for us: the `FOR...NEXT` loop.

WORKED EXAMPLE 12.05

Repetition using FOR...NEXT

The problem to be solved: Take 10 numbers as input and output the largest number.

We can use the same identifiers as in Worked Example 12.04. Note that the purpose of `Counter` has changed.

Identifier	Explanation
`BiggestSoFar`	Stores the biggest number input so far
`NextNumber`	The next number to be input
`Counter`	Counts the number of times round the loop

Table 12.07 Identifier table for biggest number problem using a `FOR` loop

The algorithm can be expressed in the following pseudocode:

```
INPUT BiggestSoFar
FOR Counter ← 2 TO 10
    INPUT NextNumber
    IF NextNumber > BiggestSoFar
      THEN
        BiggestSoFar ← NextNumber
    ENDIF
NEXT Counter
OUTPUT BiggestSoFar
```

The first time round the loop, Counter is set to 2. The next time round the loop, Counter has automatically increased to 3, and so on. The last time round the loop, Counter has the value 10.

A **rogue value** is a value used to terminate a sequence of values. The rogue value is of the same data type but outside the range of normal expected values.



KEY TERM

Rogue value: a value used to terminate a sequence of values

WORKED EXAMPLE 12.06

Repetition using a rogue value

The problem to be solved: A sequence of non-zero numbers is terminated by 0. Take this sequence as input and output the largest number.

Note: In this example the rogue value chosen is 0. It is very important to choose a rogue value that is of the same data type but outside the range of normal expected values. For example, if the input might normally include 0 then a negative value, such as −1, might be chosen.

Look at Worked Example 12.05. Instead of counting the numbers input, we need to check whether the number input is 0 to terminate the loop. The identifiers are shown in Table 12.08.

(Continued)

Identifier	Explanation
BiggestSoFar	Stores the biggest number input so far
NextNumber	The next number to be input

Table 12.08 Identifier table for biggest number problem using a rogue value

A possible pseudocode algorithm is:

```
INPUT BiggestSoFar
REPEAT
    INPUT NextNumber
    IF NextNumber > BiggestSoFar
      THEN
        BiggestSoFar ← NextNumber
    ENDIF
UNTIL NextNumber = 0
OUTPUT BiggestSoFar
```

This algorithm works even if the sequence consists of only one non-zero input. However, it will not work if the only input is 0. In that case, we don't want to perform the statements within the loop at all. We can use an alternative construct, the `WHILE...ENDWHILE` loop.

```
INPUT NextNumber
BiggestSoFar ← NextNumber
WHILE NextNumber <> 0 DO // sequence terminator not encountered
    INPUT NextNumber
    IF NextNumber > BiggestSoFar
      THEN
        BiggestSoFar ← NextNumber
    ENDIF
ENDWHILE
OUTPUT BiggestSoFar
```

Before we enter the loop, we check whether we have a non-zero number. To make this work for the first number, we store it in `NextNumber` and also in `BiggestSoFar`. If this first number is zero, we don't follow the instructions within the loop. For a non-zero first number this algorithm has the same effect as the algorithm using `REPEAT...UNTIL`.

WORKED EXAMPLE 12.07

Implementing the number-guessing game with a loop

Consider the number-guessing game again, this time allowing repeated guesses.

1. The player repeatedly inputs a number to guess the secret number stored.
2. If the guess is correct, the number of guesses made is output and the game stops.
3. If the number input is larger than the secret number, the player is given the message to input a smaller number.
4. If the number input is smaller than the secret number, the player is given the message to input a larger number.

The algorithm is expressed in structured English, as a flowchart and in pseudocode.

Algorithm for the number-guessing game in structured English:

```
SET value for secret number
REPEAT the following UNTIL correct guess
    INPUT guess
    count number of guesses
    COMPARE guess with secret number
    OUTPUT comment
OUTPUT number of guesses
```

We need variables to store the following values:

- the secret number (to be set as a random number)
- the number input by the player as a guess
- the count of how many guesses the player has made so far.

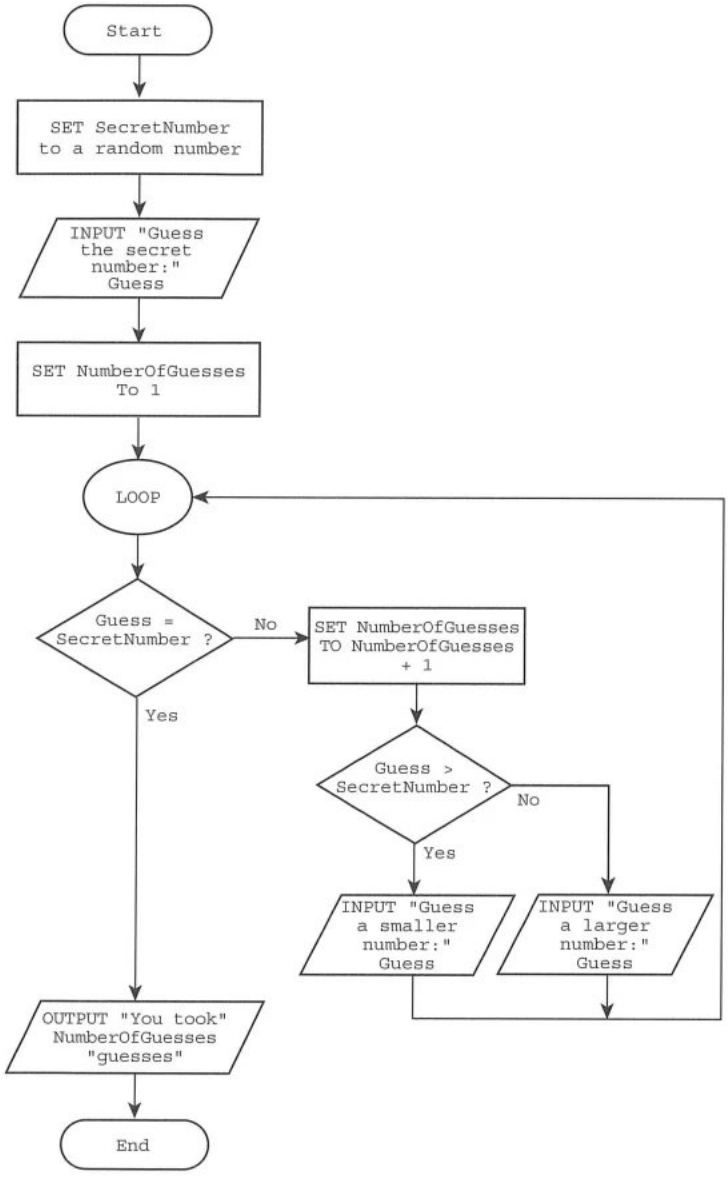
We represent this information in the identifier table shown in Table 12.09.

Identifier	Explanation
`SecretNumber`	The number to be guessed
`NumberOfGuesses`	The number of guesses the player has made
`Guess`	The number the player has input as a guess

Table 12.09 Identifier table for number-guessing game

(Continued)

Algorithm for the number-guessing game as a flowchart



Pseudocode for the number-guessing game with a post-condition loop

```
SecretNumber ← Random
NumberOfGuesses ← 0
REPEAT
    INPUT Guess
    NumberOfGuesses ← NumberOfGuesses + 1
    IF Guess > SecretNumber
```

(Continued)

```
      THEN
        // the player is given the message to input a smaller number
    ENDIF
    IF Guess < SecretNumber
      THEN
        // the player is given the message to input a larger number
    ENDIF
UNTIL Guess = SecretNumber
OUTPUT NumberOfGuesses
```

Pseudocode for the number-guessing game with a pre-condition loop

```
SecretNumber ← Random
INPUT Guess
NumberOfGuesses ← 1
WHILE Guess <> SecretNumber DO
    IF Guess > SecretNumber
      THEN
        // the player is given the message to input a smaller number
    ENDIF
    IF Guess < SecretNumber
      THEN
        // the player is given the message to input a larger number
    ENDIF
    INPUT Guess
    NumberOfGuesses ← NumberOfGuesses + 1
ENDWHILE
OUTPUT NumberOfGuesses
```


WORKED EXAMPLE 12.08

Calculating running totals and averages

The problem to be solved: Take 10 numbers as input and output the sum of these numbers and the average.

Identifier	Explanation
RunningTotal	Stores the sum of the numbers input so far
Counter	How many numbers have been input
NextNumber	The next number input
Average	The average of the numbers input

Table 12.10 Identifier table for running total and average algorithm

The following pseudocode gives a possible algorithm:

```
RunningTotal ← 0
FOR Counter ← 1 TO 10
    INPUT NextNumber
    RunningTotal ← RunningTotal + NextNumber
NEXT Counter
OUTPUT RunningTotal
Average ← RunningTotal / 10
OUTPUT Average
```

(Continued)

It is very important that the value stored in `RunningTotal` is initialised to zero before we start adding the numbers being input.

TIP

Which type of loop? If it is known how many repetitions are required, choose a FOR loop. If the statements inside the loop might never be executed, choose a WHILE loop. If the statements inside the loop are to be executed at least once, a REPEAT loop might be more sensible.

TASK 12.05

Change the algorithm in Worked Example 12.08 so that the sequence of numbers is terminated by a rogue value of 0.

WORKED EXAMPLE 12.09

Using nested loops

The problem to be solved: Take as input two numbers and a symbol. Output a grid made up entirely of the chosen symbol, with the number of rows matching the first number input and the number of columns matching the second number input.
For example the three input values 3, 7 and &, result in the output:

```
&&&&&&&
&&&&&&&
&&&&&&&
```

We need two variables to store the number of rows and the number of columns. We also need a variable to store the symbol. We need a counter for the rows and a counter for the columns.

Identifier	Explanation
`NumberOfRows`	Stores the number of rows of the grid
`NumberOfColumns`	Stores the number of columns of the grid
`Symbol`	Stores the chosen character symbol
`RowCounter`	Counts the number of rows
`ColumnCounter`	Counts the number of columns

Table 12.11 Identifier table for the nested loop example

```
INPUT NumberOfRows
INPUT NumberOfColumns
INPUT Symbol
FOR RowCounter ← 1 TO NumberOfRows
    FOR ColumnCounter ← 1 TO NumberOfColumns
        OUTPUT Symbol // without moving to next line
    NEXT ColumnCounter
    OUTPUT Newline   // move to the next line
NEXT RowCounter
```

Each time round the outer loop (counting the number of rows) we complete the inner loop, outputting a symbol for each count of the number of columns. This type of construct is called a **nested loop**.